Day 23 — IDoc Splitter, Other Splitters, Gather & Join

Part A — Understanding the IDoc

1. What Is an IDoc?

> **IDoc = Intermediate Document.** A **standard document / standard data structure** used to **transfer data between one SAP system and another system** — including non-SAP systems. It is a **standard format of structure**; in CPI it is carried as a **standard XML-format content**. It is a **concept of SAP** — it exists in **ECC and S/4HANA**.

Example — Purchase Order IDoc

- Create a purchase order (in ECC or S/4HANA) → it can be sent out as an **XML payload** in a **standard format**.
- **Scenario:** SAP defines a **standard purchase-order IDoc structure**. Now if anyone wants to **send purchase-order data into SAP ECC / S/4HANA**, they take the **existing standard purchase-order IDoc structure**, fill it with their purchase-order details, and send it to the SAP system → the data gets added.

> **Correction — "IDoc will always be in XML file only":** this is true *for what you handle in CPI* — the IDoc adapter exchanges IDocs as **IDoc-XML**. But inside an SAP system an IDoc is stored as **records** (control, data and status records in tables), and it can also exist as a flat file. So the accurate statement is: "*in CPI, an IDoc is an XML message.*"
>
> **Also:** IDocs are used for SAP ↔ non-SAP **and** SAP ↔ SAP exchange (ALE for master data, EDI for transactions).

2. IDoc Structure — Segments, Records & the Control Record

An IDoc is made of **segments (nodes)**. **Every IDoc starts with an IDOC tag:**

```
<ORDERS05>
  <IDOC BEGIN="1">
    <EDI_DC40 SEGMENT="1">   ← control record
      ...
    </EDI_DC40>
    <E1EDK01 SEGMENT="1"> ... </E1EDK01>   ← data segments
    ...
  </IDOC>
</ORDERS05>
```

The 3 Mandatory Record Types

#	Record	What It Holds
①	**Control record** (`EDI_DC40`)	The header information — the **mandatory segment** that describes the IDoc
②	**Data records**	**All segments below the control record** — the actual business data
③	**Status records**	The processing status history of the IDoc

> **Practical note:** status records are created **inside the SAP system** as the IDoc is processed, so the XML you handle in CPI normally shows just the **control record + data records**.

What the Control Record Tells You

#	Information	Typical Field
①	What is the **IDoc type**	`IDOCTYP` (e.g., ORDERS05)
②	What **message** they are passing (message type)	`MESTYP` (e.g., ORDERS)
③	Where the IDoc is **coming from**	`SNDPRN` / `SNDPOR` (sender)
④	Where the IDoc is **going to**	`RCVPRN` / `RCVPOR` (receiver)

⑤	The **IDoc number** (unique)	`DOCNUM`
⑥	The **status** of the IDoc (successful or failed)	`STATUS`

3. Standard vs. Custom IDocs

- SAP uses IDocs mainly for **master data** (and transactional data).
- Some organizations have **additional (extra-field) requirements**, so they **extend the standard IDoc**.

Standard IDoc	Custom IDoc
`<ORDERS05>`	**Starts with Z** — e.g., `<ZORDERS05>`

Mapping to/from an IDoc

When you send an IDoc through **message mapping**, **we don't write the IDoc structure manually** — instead we **import** the IDoc structure into the mapping (Transformations → Message Mapping), so every segment and field name is exactly as SAP defines it.

> **One line I couldn't interpret:** your notes say **"Document Segment → mandatory ✓"**. It may refer to the mandatory control segment (`EDI_DC40`) or to the `IDOC` wrapper element — I'd rather flag it than guess. Ask your trainer which one was meant.

Part B — The Other Splitters

4. IDoc Splitter

> **Why the IDoc Splitter exists:** when you get **multiple IDocs at a time** and you want to send **a single IDoc at a time**, you use the **IDoc Splitter**.

- It works on **incoming (sender) IDocs**, because the input is **several IDocs in a single payload**, and you want to split that message into multiple messages.
- **IDoc packaging → grouping of IDocs:** the sending SAP system can bundle several IDocs into one package — that bundle is what arrives as one composite message.
- **No configuration properties** are needed in the IDoc Splitter — it automatically **differentiates the IDocs based on** `<IDOC BEGIN="1">` .

> **Verified in SAP's documentation:**
>
> - "No parameters are to be configured for this step." It works **exactly like a General Splitter with the XPath expression** `//IDOC` — it splits at the `IDOC` element.
> - **Each individual message keeps the enveloping elements** of the composite message (like the General Splitter, Day 22), so each output still looks like a complete `<ORDERS05><IDOC>…</IDOC></ORDERS05>` .
> - **Bonus headers** you can use afterwards: `SapIDocTransferId` (the original incoming IDoc number, i.e. DOCNUM), `SapIDocDbId` , and `SapIDocAssignMap` (the IDoc numbers returned from the IDoc receiver).
> - Because it simply splits at `IDOC` , it works on any composite IDoc-XML — typically it is used with IDocs coming from the IDoc sender adapter.

5. PKCS#7/CMS, Zip, Tar and EDI Splitters

Splitter	From Class	Verified Detail
PKCS#7/CMS Splitter	"Public-key based encrypted messages we want to split"	Takes a **CMS SignedData** message and splits it into **two messages: the payload and its signature** (it converts a normal signed message to "detached" mode). You can name the payload file and signature file.
Zip Splitter	Discussed together with Tar	Splits a **ZIP archive** into its individual files (one message per file)
Tar Splitter	Same functionality as Zip	Splits a **TAR archive** into its individual files
EDI Splitter	EDIFACT, B2B	Splits an **EDI interchange** (e.g., EDIFACT) into separate EDI messages; used with

	scenarios	schemas from Integration Advisor

Refinements:

- **PKCS#7/CMS is about signatures, not general encryption:** it separates the **payload from its digital signature**. (Signatures use public-key cryptography, which is probably why "public key" came up in class.)
- **Zip and Tar behave alike** (archive → files). The PKCS#7/CMS splitter does a **different job** — your notes say its functionality is the same as Zip/Tar; it isn't, it just also produces split messages.
- **EDI Splitter caution:** a SAP Community answer notes the EDI Splitter does **not** work with the standard Gather step — to collect results after an EDI split you may need a script or sequential processing.

Part C — Combining Messages: Gather & Join

6. Gather

So far we have been **splitting** data. If we split the data, we **need to combine** it as well, right?

> **Gather:** to **combine multiple messages into a single message**, we use Gather.
> **Scenario:** Splitter (or Multicast) → multiple messages → combine → **single message** → **Gather**.

Gather Settings (Added, Verified)

Setting	Options
Incoming Format	XML (Same Format) · XML (Different Format) · Plain Text · Any
Aggregation Algorithm	**Combine** (XML only — appends the messages into a multi-message payload) · **Combine at XPath** (XML Same Format only — merges at a node you choose) · **Concatenate** (Plain Text only — joins the texts one after another)

7. Join

With Gather we have **one more palette function: Join.**

> **Join:** to **combine the data coming from multiple branches into a single branch.**

8. Difference Between Gather and Join

Both functionalities involve "combining" — but **what exactly does each one do?**

<table>
<tr><td rowspan="2">Start</td><td rowspan="2">→ CM →</td><td rowspan="2">Parallel Multicast</td><td>Branch 1 → CM</td><td rowspan="2">→ Join → Gather →</td><td rowspan="2">End</td></tr>
<tr><td>Branch 2 → CM</td></tr>
</table>

What happened in class: with a Multicast and two branches, placing **only a Gather** after the branches gave **no output** — **because Gather takes its input from one branch only (remember — important).** After adding a **Join** in front of it, it worked.

	Join	Gather
Job	**Only combines the branches** into one path — nothing else; it doesn't change any message content	**Combines the messages** into one single message (using the chosen strategy)
Input	Several branches (from a Multicast)	One incoming path (so after a Multicast it needs a Join first)
Used with	Multicast (always paired with Gather)	Splitter (alone) or Multicast (with Join before it)

> **Golden rules from class:**
>
> - **Multicast → Join → Gather** (a Gather alone won't work after a Multicast).

- **If we place a Content Modifier between Join and Gather, it will not work** — keep Gather **directly after** Join (put extra steps in the branches before the Join, or after the Gather).
- **Splitter → … → Gather** needs no Join, because a Splitter keeps everything on one path.

Verified in SAP's documentation: "*If you want to combine messages that are transmitted to more than one route by a Multicast step, you need to place a Join step before the Gather step.*" SAP also describes Join as bringing together messages from different routes **without affecting their content**, and without a Join, every branch would continue through the remaining steps on its own.

Watch-outs (from SAP guidance and community reports):

- **If one branch fails in a Join + Gather setup, processing stops** — unlike plain Parallel Multicast (Day 21), where the other branches carry on.
- **Headers and properties after Gather depend on the algorithm.** A community test found: with **Combine**, only branch 1's properties/headers survive; with **Combine at XPath**, headers and properties from **all** branches are kept. If you need values from every branch after the merge, test your setting with Trace.

Don't Confuse Gather with Aggregator

Day 9 also listed **Aggregator**. **Gather** merges the messages created **within one run** (from a Splitter or Multicast). The **Aggregator** collects **separate incoming messages over time** (e.g., several messages from the same sender) until a completion condition (such as a timeout) is met. Different jobs.

9. Quick Interview Answers

Question	Short Answer
What is an IDoc?	Intermediate Document — SAP's standard structure for exchanging data; carried as XML in CPI
What are the 3 record types?	Control record, data records, status records
Standard vs. custom IDoc?	Standard e.g. ORDERS05; custom starts with Z, e.g. ZORDERS05
Why the IDoc Splitter?	Several IDocs arrive in one message and you need one at a time
How is it configured?	No parameters — it splits at the IDOC element, like a General Splitter on //IDOC
What does the PKCS#7/CMS Splitter do?	Separates a signed message's payload from its signature
Gather vs. Join?	Join only merges the branches into one path; Gather merges the messages into one message
Why was there no output with Gather after a Multicast?	Gather takes input from one path — a Join must come first
Can I put a Content Modifier between Join and Gather?	No — Gather must follow Join directly